EXPERIMENT 06

Aim: Clustering using K-Means and Hierarchical Clustering

Objectives:

1. To understand the working principles of K-Means and Hierarchical Clustering algorithms.
2. To preprocess the dataset using feature scaling.
   To implement clustering models using Python's **scikit-learn** library.
3. To visualize clusters and evaluate their quality.

Theory:

Clustering is an **unsupervised learning technique** used to group similar data points together based on their features. Unlike classification, clustering does not use labeled data.

Two widely used clustering algorithms are:

- **K-Means Clustering** – partitions data into k clusters by minimizing intra-cluster variance.
- **Hierarchical Clustering** – builds clusters in a tree-like structure (dendrogram).

The implementation of clustering algorithms generally follows three main stages:

1. Data Preparation
2. Model Creation
3. Cluster Evaluation

Python's **scikit-learn** library provides efficient tools for clustering.

Sample Dataset (Used for KNN Classification):

A synthetic **customer segmentation dataset** is used.
Each record represents a customer with purchasing behavior

features. **Features:**

- Annual Income
- Spending Score
- Savings
  Online Shopping Frequency

Why Is This Dataset Suitable for Clustering?

- Contains multiple numerical features ideal for grouping
- No predefined labels (unsupervised learning)
- Demonstrates real-world customer segmentation
- Requires feature scaling for better clustering results

Features Description

Feature Name	Type	Description
`Income`	Numeric	Annual income of the customer in USD
`Spending_Score`	Numeric	Spending score based on purchasing habits (0-100)
`Savings`	Numeric	Total estimated savings of the customer in USD
`Online_Shopping_Frequency`	Numeric	Number of online purchases per month
`Cluster_Label`	Categorical	Cluster assignment (used after applying K-Means or Hierarchical Clustering)

Sample Dataset Table

Income	Spending_Score	Savings	Online_Shopping_Frequency
52000	75	15000	12
40000	45	8000	5
60000	90	25000	18
30000	30	5000	2
75000	95	40000	20

1. Prepare and Preprocess the Data

1.1 Load the Dataset: Load dataset using pandas DataFrame.

1.2 Feature Selection: Use all features since clustering has no target variable.

1.3 Feature Scaling: Important because clustering depends on distance, using StandardScaler

2. Implement the Clustering Models:

2.1 K-Means Clustering: Ensures to Initialize Model**.** Choose number of clusters
Train Model: Use fit_predict() to assign clusters

2.2 Hierarchical Clustering: To ensure and Initialize Model. Using agglomerative clustering.
Train Model: Assign clusters using linkage method

3. Evaluate Model Performance

Evaluating performance involves comparing **predicted values** with **actual test values**.

3.1 Evaluation Metrics

- **Inertia (K-Means)**
  Measures compactness of clusters
- **Silhouette Score**
  Measures how well-separated clusters are

3.2 Elbow Method (For Optimal k)

- Plot number of clusters vs inertia
- Choose point where curve bends (elbow point)

Code:

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.preprocessing import StandardScaler from
sklearn.cluster import KMeans, AgglomerativeClustering from
sklearn.metrics import silhouette_score
from scipy.cluster.hierarchy import dendrogram, linkage

# Generate synthetic dataset
np.random.seed(42)
data_size = 300

data = {
 "income": np.random.normal(50000, 15000, data_size),
```

```
"spending_score": np.random.normal(50, 20, data_size),
"savings": np.random.normal(20000, 10000, data_size),
"online_freq": np.random.normal(10, 5, data_size) }

df = pd.DataFrame(data)
print(df.head())

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

# -----------------------------------
# K-MEANS CLUSTERING
# -----------------------------------

# Elbow Method
inertia = []
k_values = range(1, 11)

for k in k_values:
 kmeans = KMeans(n_clusters=k, random_state=42)
kmeans.fit(X_scaled)
 inertia.append(kmeans.inertia_)

# Plot Elbow Graph
plt.figure(figsize=(8,5))
plt.plot(k_values, inertia, marker='o')
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.title("Elbow Method for Optimal k")
plt.grid(True)
plt.show()

# Optimal k (example)
k_opt = 4

kmeans = KMeans(n_clusters=k_opt, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Silhouette Score
print("K-Means Silhouette Score:", silhouette_score(X_scaled,
kmeans_labels))


# -----------------------------------
# HIERARCHICAL CLUSTERING
```

```
# ----------------------------------

# Dendrogram
linked = linkage(X_scaled, method='ward')

plt.figure(figsize=(10,5))
dendrogram(linked)
plt.title("Dendrogram")
plt.xlabel("Data Points")
plt.ylabel("Distance")
plt.show()

# Apply Hierarchical Clustering
hc = AgglomerativeClustering(n_clusters=4)
hc_labels = hc.fit_predict(X_scaled)

# Silhouette Score
print("Hierarchical Clustering Silhouette Score:",
silhouette_score(X_scaled, hc_labels))
```

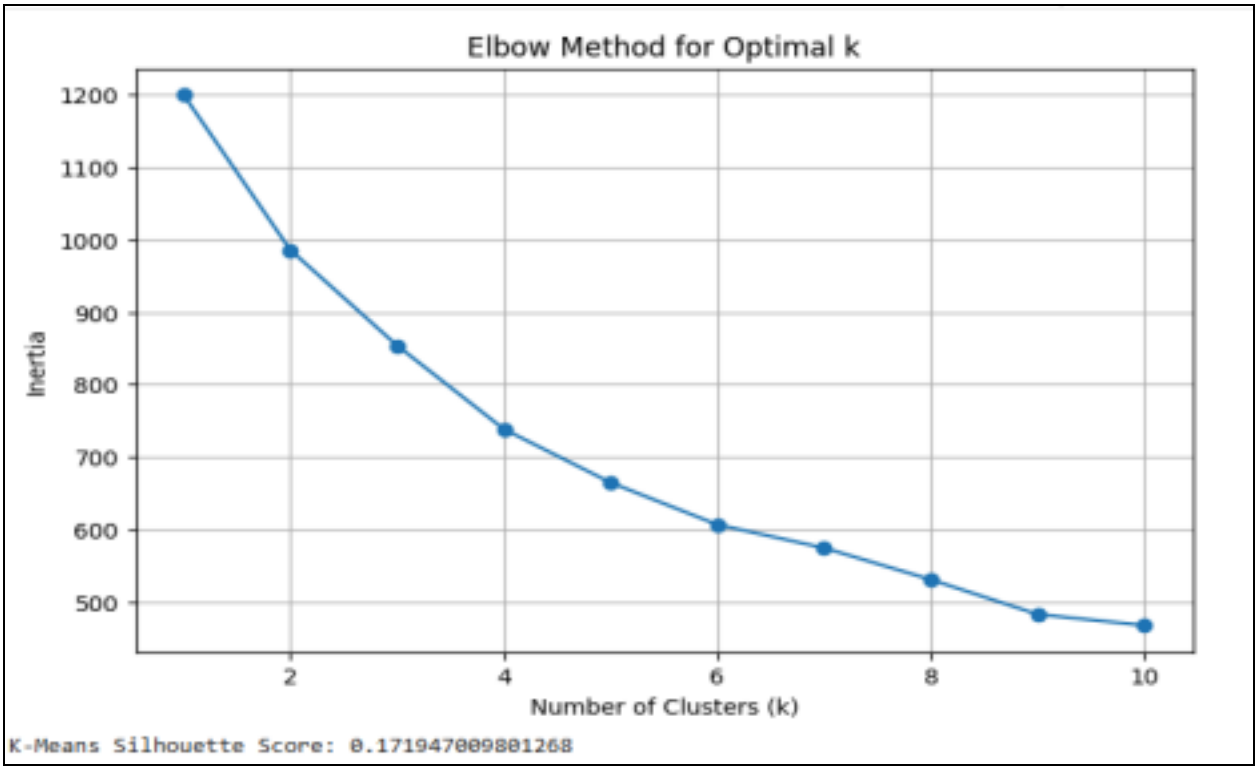
Output:

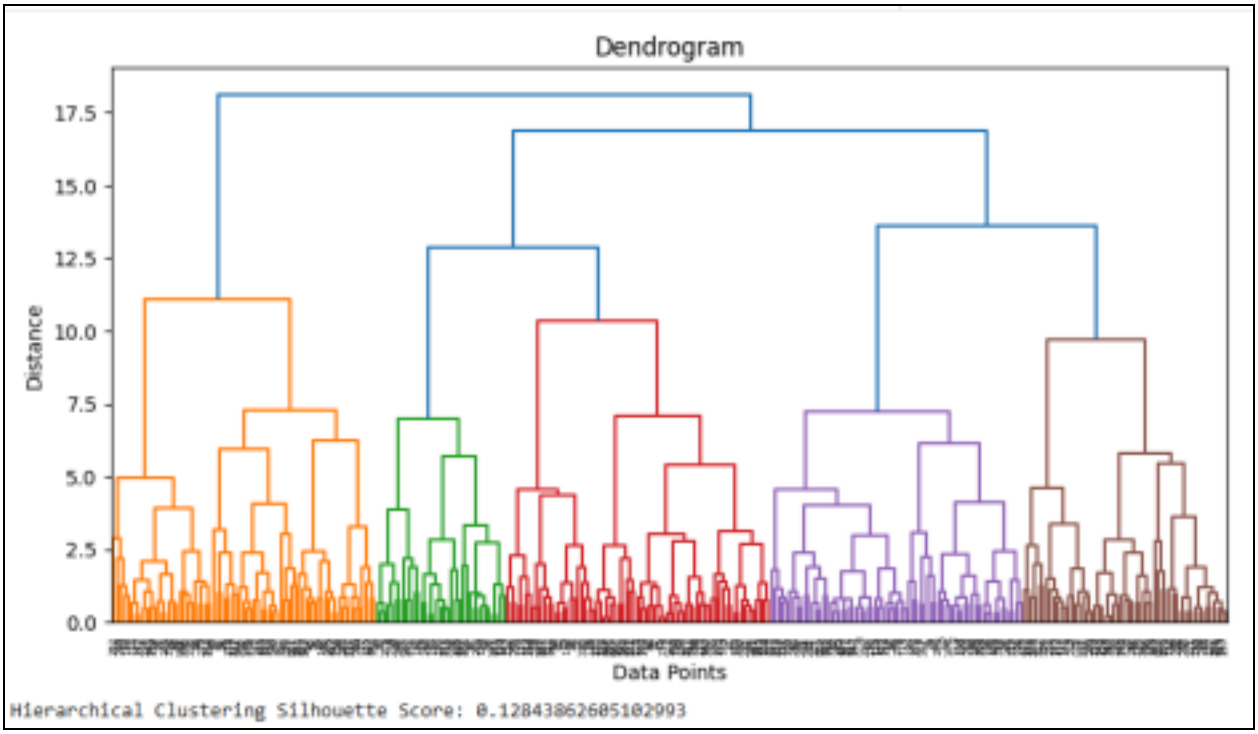
```
data = {
    "income": np.random.normal(50000, 15000, data_size),
    "spending_score": np.random.normal(50, 20, data_size),
    "savings": np.random.normal(20000, 10000, data_size),
    "online_freq": np.random.normal(10, 5, data_size)
}

df = pd.DataFrame(data)
print(df.head())

# Feature Scaling
scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)
```

```
...          income  spending_score       savings  online_freq
0  57450.712295       33.420100  27569.886166    11.843367
1  47926.035482       38.796379  10778.346758     8.033306
2  59715.328072       64.945872  28696.059201    10.143724
3  72845.447846       62.207405  33556.378588    16.392259
4  46487.699379       49.581968  24134.349032    10.955495
```





The **Elbow graph** shows the optimal number of clusters around **k = 4**.
The **Dendrogram** visually represents how clusters merge at different distances.

Silhouette scores indicate how well clusters are separated:

Higher score → better clustering

Conclusion:

K-Means and Hierarchical Clustering algorithms were successfully implemented for customer segmentation. Feature scaling significantly improved clustering performance. The Elbow method helped identify the optimal number of clusters, while the dendrogram provided a visual understanding of hierarchical relationships. Both algorithms effectively grouped similar data points, demonstrating their usefulness in unsupervised learning tasks